Krogh GUI
Publication-Level Mathematical, Numerical, and Implementation Documentation

Technical reconstruction from the current source code

18 April 2026

Contents

1 Abstract **3**

2 Current project state after the April 2026 refactor **3**

2.1 What has already been separated into modules 3

2.2 What still remains in the main GUI file 4

2.3 Practical consequence for users and reviewers 4

2.4 Snapshot of the actual architecture 4

2.5 Most important verified extensions from the latest work cycle 5

3 Scientific scope of the application **5**

4 Mathematical formulation **6**

4.1 Krogh cylinder geometry 6

4.2 Bohr effect, carbon dioxide effect, and temperature shift 6

4.3 Hemoglobin saturation 6

4.4 Differential oxygen capacity 6

4.5 Radial Krogh-Erlang solution 7

4.6 Michaelis-Menten-like oxygen consumption 7

5 From mathematical theory to Python functions **7**

6 Exact numerical workflow in the code **8**

6.1 Grid generation 8

6.2 Step 1: self-consistent consumption for a given capillary PO_2 8
6.3 Step 2: axial capillary transport ODE 8
6.4 Step 3: axial-radial tissue field with finite differences 9
6.5 Step 4: outer capillary-tissue coupling 9

7 How outputs are generated from the solved fields 9

8 Important numerical assumptions and their implications 10
8.1 Why the model fits robustly in many cases 10
8.2 When stricter settings are scientifically advisable 10

9 Diagnostic layer and inverse reconstruction 11
9.1 From venous saturation to a fitting target 11
9.2 Joint fitting of perfusion and mitochondrial demand 11
9.3 Why different venous weights are used 12
9.4 How fit quality is judged 12
9.5 What the optimized diagnostic workflow now reports 12

10 Reporting and export workflow 12

11 Libraries and their concrete roles 13

12 Interpretation guidelines for publication use 13

13 Limitations 13

14 Conclusion 14

1 Abstract

This document provides an English, publication-oriented description of the application implemented in `krogh_GUI.py`. Its goal is not merely to summarize the user interface, but to establish a traceable chain from physiological theory to mathematical equations, from equations to numerical algorithms, and from numerical algorithms to the exact Python functions that produce the reported outputs.

Special emphasis is placed on transparency. The document explains which library routines are called, at what stage of the computation they are used, what their most important parameters are, and which quantities they return. This is particularly important for the diagnostic and reconstruction workflow, where model-based fitting must remain auditable if a reviewer later asks why a certain result was obtained.

2 Current project state after the April 2026 refactor

The software is no longer just one monolithic Tkinter script. In the current verified state, the project follows a hybrid architecture: the validated numerical core remains directly readable, while orchestration, plotting, persistence, diagnostics, localization, and UI helpers have been extracted into a dedicated package called `krogh_app`.

The user-facing entry point is now `app.py`. This file is intentionally small and only launches the main window. The file `krogh_GUI.py` still exists, but now acts primarily as a compatibility-oriented coordination layer so that established workflows remain intact while the internals become easier to maintain.

2.1 What has already been separated into modules

At the present phase of development, the following responsibilities live outside the legacy GUI shell:

- `krogh_app.constants`: physical constants, discretization defaults, and numeric configuration ranges,
- `krogh_app.types`: structured dataclasses for simulation inputs and diagnostic runs,
- `krogh_app.diagnostics`: the diagnostic engine wrapper for the oxygenation MVP logic,
- `krogh_app.reconstruction`: inverse fitting and preparation of reconstruction plot data,
- `krogh_app.series`: one-dimensional and two-dimensional sweep execution plus numeric sensitivity checks,
- `krogh_app.plotting`: reusable plot styling and workflow coordinators for line plots, heatmaps, and 3D surfaces,
- `krogh_app.persistence`: capture and restoration of GUI state as well as run export support,

- `krogh_app.helptext`: generation of explanatory output and numerics help text,
- `krogh_app.localization`: centralized translation, label formatting, and multilingual text handling,
- `krogh_app.ui.*`: focused coordinators for layout, runtime updates, execution control, figure display, dialogs, and tooltips.

2.2 What still remains in the main GUI file

The current `krogh_GUI.py` still contains the validated mechanistic solver functions, the translation dictionaries, and the top-level `KroghGUI` class. This is intentional. The refactor has been performed incrementally in order to preserve the mathematically validated behaviour while reducing structural complexity step by step rather than rewriting the application all at once.

2.3 Practical consequence for users and reviewers

For everyday use, the graphical workflow is intentionally almost unchanged. For maintenance and scientific auditability, however, the project is now much clearer: individual responsibilities can be inspected in small modules, numerical behaviour can be checked independently of the UI, and future work can extend isolated services without destabilizing the whole program.

2.4 Snapshot of the actual architecture

Current file or package area	Current role in the software
`app.py`	Minimal startup entry point that launches the GUI.
`krogh_GUI.py`	Compatibility host and central controller; still contains the scientific core functions and the main Tkinter window class.
`krogh_app/plotting.py`	Shared figure styles plus the coordinators for series plots and 3D rendering.
`krogh_app/series.py`	Parameter sweep generation, execution, and numerics comparison logic.
`krogh_app/reconstruction.py`	Joint fitting from diagnostic values back to plausible Krogh parameters.
`krogh_app/diagnostics.py`	Stable wrapper around the probabilistic oxygenation diagnostic logic.
`krogh_app/persistence.py`	Save/load of cases and export-related state handling.
`krogh_app/localization.py`	TranslationManager for multilingual labels, state names, and result text.
`krogh_app/ui/`	Small single-responsibility UI helpers for layout, runtime state, dialogs, execution, and figures.

In summary, the project is currently in a productive transitional state: scientifically operational, clinically interpretable, and already substantially modularized, while still preserving the legacy entry points needed for continuity.

2.5 Key verified improvements from the latest development cycle

The documentation should explicitly reflect that the project was not only reorganized structurally, but also strengthened scientifically and clinically during the most recent development cycle. The key verified additions are:

- the inverse reconstruction now reports a practical near-optimal uncertainty band for mitoP50 and perfusion instead of only a single best-fit point,
- diagnostic and reconstruction results can now be exported directly as structured JSON/CSV reports,
- the GUI can generate an English publication report as plain text, LaTeX, or PDF, and the latest reconstruction figure is embedded automatically when available,
- the publication report now contains explicit narrative sections for probabilistic result, clinical interpretation, model-based findings, cautions and limitations, and suggested follow-up,
- the diagnostic output identifies the dominant alert-score drivers so that the color category is not a black-box label,
- the reconstruction quantifies hidden hypoxic burden below 1, 5, 10, 15, and 20 mmHg instead of relying only on a mean tissue value,
- geometric uncertainty is now addressed more transparently by comparing radius assumptions of 30, 50, and 100 µm, and the diagnostic tab can evaluate either all variants or one user-selected radius,
- the report wording now states explicitly when an elevated alert is supported chiefly under the larger or swollen-tissue geometry, which makes the clinical relevance easier to judge.

3 Scientific scope of the application

The software combines four layers in a single environment:

1. simulation of a single Krogh-cylinder oxygenation case,
2. one-dimensional and two-dimensional parameter sweeps,
3. user-adjustable numerical solver control,
4. probabilistic diagnostics and inverse reconstruction from measurement-like inputs.

In scientific terms, the program is a mechanistic microcirculatory oxygen transport model with an attached diagnostic interpretation layer. In engineering terms, it is a transparent computational workflow in which the physiological assumptions are explicit and the numerical path to the final outputs can be inspected.

4 Mathematical formulation

4.1 Krogh cylinder geometry

The model idealizes one capillary as the axis of a tissue cylinder. The capillary radius is denoted by R_{cap} and the tissue radius by R_{tis}. Oxygen is transported axially in blood and diffuses radially into tissue, where it is consumed by metabolism.

The central unknowns are the capillary oxygen partial pressure $P_c(z)$ and the tissue oxygen field $P(z, r)$.

4.2 Bohr effect, carbon dioxide effect, and temperature shift

The code first transforms the standard hemoglobin half-saturation pressure P_{50} into an effective value:

$$P_{50,eff} = P_{50} \cdot 10^{\beta_{pH}(pH - PH_{ref}) + \beta_{CO_2} \log_{10}(pCO_2/PCO2_{ref}) + \beta_T(T - T_{ref})}.$$

This relation is implemented by the function `effective_p50(pH, pco2, temp_c)`. Numerically, the code protects the logarithm by replacing very small carbon dioxide values with `max(pco2, 1e-6)`. This simple safeguard prevents undefined values during diagnostic or extreme-case runs.

4.3 Hemoglobin saturation

The oxygen saturation curve follows the Hill relation

$$S(P) = \frac{P^n}{P^n + P_{50,eff}^n},$$

with Hill exponent $n = 2.7$ in the current code base. The function `hill_saturation(P, p50_eff=None)` computes this quantity for scalar or array input. The implementation applies `np.maximum(P, 1e-9)` so that the power law remains numerically stable even close to zero oxygen tension.

4.4 Differential oxygen capacity

The capillary pressure equation requires the derivative of oxygen content with respect to oxygen partial pressure:

$$\frac{dC}{dP} = C_{Hb}\frac{dS}{dP} + \alpha.$$

This term is computed in `dC_dP(P, p50_eff=None)`. It is a key nonlinearity: the same oxygen extraction rate causes different pressure drops depending on whether the local point lies in a steep or flat segment of the saturation curve.

4.5 Radial Krogh-Erlang solution

For a fixed axial position and an effective metabolic demand M, the code uses the classical Krogh-Erlang expression

$$P(r) = P_c + \frac{M}{4K}(r^2 - R_c^2) - \frac{MR_t^2}{2K}\ln\left(\frac{r}{R_c}\right).$$

This is implemented in `krogh_erlang(r, P_c, M, K, R_c, R_t)` and serves both as a direct estimate and as an initialization profile for the coupled two-dimensional solve.

4.6 Michaelis-Menten-like oxygen consumption

Tissue consumption is not constant. Instead, the code uses

$$M(P) = M_{max}\left[f_{min} + (1 - f_{min})\frac{P}{P + P_{half}}\right].$$

The function `michaelis_menten_consumption(P_local, P_half, M_max=M_rate)` implements this expression. The parameter P_{half} controls how strongly metabolism becomes oxygen-limited. The constant `MIN_CONSUMPTION_FRACTION` ensures that consumption does not collapse to an artificial zero at extremely low oxygen values.

5 From mathematical theory to Python functions

The following table is the core translation layer between the theory and the implementation.

Mathematical object	Python function	Main inputs	Returned quantity
Bohr/CO_2/temp. correction	`effective_p50`	`pH`, `pco2`, `temp_c`	Effective P_{50} (mmHg)
Hill saturation	`hill_saturation`	`P`, optional `p50_eff`	Saturation in $[0, 1]$
Content derivative	`dC_dP`	`P`, optional `p50_eff`	Capacity term for the ODE
Radial analytic profile	`krogh_erlang`	Radius vector, capillary boundary, demand, diffusion coefficient	Radial tissue PO_2 profile
Nonlinear metabolic law	`michaelis_menten_consumption`	Local PO_2, `P_half`	Consumption field or scalar
Self-consistent mean demand	`effective_consumption_from_capillary_po2`	Capillary PO_2, `P_half`	Effective axial demand
Capillary ODE solver	`solve_capillary_profile`	ODE right-hand side and inlet PO_2	Axial capillary profile
Two-dimensional diffusion solve	`solve_tissue_field_with_axial_diffusion`	Capillary profile, `P_half`, optional initial guess	Tissue field over (z, r)
Coupled tissue-capillary solve	`solve_axial_capillary_po2`	Inlet PO_2, `P_half`, `p50_eff`, diffusion flag, perfusion factor	Capillary profile, tissue field, mean demand
User-facing model run	`run_single_case`	Physiological and threshold inputs	Final metrics dictionary

6 Exact numerical workflow in the code

6.1 Grid generation

At import time, the code builds the numerical mesh using NumPy:

- `z_eval = np.linspace(0, L_cap, NZ)` for the axial positions,
- `r_vec = np.linspace(R_cap, R_tis, NR)` for the radial positions,
- `dr` and `dz` from the mesh spacing,
- `radial_weights = r_vec / np.sum(r_vec * dr)` for weighted averaging.

This weighting is important: the model does not treat each radial node as equally representative, but approximately respects the cylindrical area element.

6.2 Step 1: self-consistent consumption for a given capillary PO_2

The function `effective_consumption_from_capillary_po2(P_c, P_half, max_iter=30, tol=1e-7)` performs a fixed-point iteration:

1. start from `M_eff = M_rate`,
2. compute a radial profile by calling `krogh_erlang`,
3. clip negative values through `np.maximum(profile, 0.0)`,
4. compute the weighted mean by `np.average(profile, weights=radial_weights)`,
5. evaluate a new metabolic demand via `michaelis_menten_consumption`,
6. stop when the absolute change drops below `tol`.

This is not a black box optimization. It is an explicit, auditable fixed-point update.

6.3 Step 2: axial capillary transport ODE

The capillary transport equation is solved as

$$\frac{dP_c}{dz} = -\frac{M_{eff}(z)\pi R_{tis}^2}{Q_{local}\, dC/dP}.$$

The corresponding Python routine is `solve_capillary_profile(dPc_dz, P_inlet)`. Internally it calls:

```
solve_ivp(dPc_dz, (0, L_cap), [P_inlet], t_eval=z_eval,
method="RK45", rtol=CAPILLARY_ODE_RTOL, atol=CAPILLARY_ODE_ATOL,
max_step=CAPILLARY_ODE_MAX_STEP)
```

Thus the main imported SciPy routine is `scipy.integrate.solve_ivp`, used with the adaptive explicit Runge-Kutta method `RK45`. The default tolerances in the code are tight `(rtol = 1e-9, atol = 1e-11)`, and the maximum step is tied to the axial mesh width.

6.4 Step 3: axial-radial tissue field with finite differences

If axial diffusion is activated, the program enters the following function:

```
solve_tissue_field_with_axial_diffusion(P_c_axial, P_half, initial_guess=None)
```

Here the tissue field is discretized by finite differences. The update coefficients

$$a_+,\ a_-,\ a_z,\ \text{and denom}$$

are derived from the diffusion coefficient `K_diff` and the mesh widths `dr` and `dz`.

At every inner iteration the code computes the local consumption field from the previous iterate, then updates the interior nodes, then applies under-relaxation with factor `AXIAL_DIFFUSION_RELAX = 0.45`. The iteration terminates when

$$\max |P^{new} - P^{old}| < \texttt{AXIAL_DIFFUSION_TOL}.$$

6.5 Step 4: outer capillary-tissue coupling

The function `solve_axial_capillary_po2(...)` wraps the entire coupled calculation. If axial diffusion is enabled, the algorithm alternates between:

1. updating the tissue field,
2. computing the slice-wise average demand by `np.average(..., axis=1, weights=radial_weights)`,
3. interpolating the new demand along the capillary with `np.interp(z_pos, z_eval, M_slice)`,
4. solving the capillary ODE again.

The outer loop stops when the maximum absolute difference between two consecutive capillary profiles is below `AXIAL_COUPLING_TOL`. This splitting strategy is computationally transparent and is easier to diagnose than a monolithic nonlinear solver.

7 How outputs are generated from the solved fields

The single-case entry point is `run_single_case(P_inlet, P_half, pH, pCO2, temp_c, perfusion_factor, include_axial_diffusion, ...)`. Its output is a metrics dictionary whose contents are derived as follows:

- `P_venous`: final element of the capillary profile, i.e. `float(P_c_axial[-1])`,
- `P_tissue_min`: global minimum of the tissue field,
- `P_tissue_p05`: 5th percentile computed by `np.percentile(tissue_values, 5.0)`,

- hypoxic fractions: percentages below 1, 5, or 10 mmHg computed by `np.mean(tissue_values < threshold)`,
- high-oxygen fractions: percentages above absolute or relative thresholds,
- `PO2_sensor_avg`: axial mean of the weighted radial average,
- `S_a` and `S_v`: arterial and venous saturation obtained from `hill_saturation`.

Therefore, every user-visible number is directly linked to a compact NumPy operation or to a named model function. This is precisely the kind of traceability required for publication-quality reporting.

8 Important numerical assumptions and their implications

8.1 Why the model fits robustly in many cases

The program uses several stabilizing choices:

1. clipping negative PO_2 values to zero,
2. enforcing a strictly positive perfusion denominator via `max(perfusion_factor * Q_flow, 1e-12)`,
3. protecting logarithms and power laws from singular input,
4. starting the coupled problem from analytic Krogh profiles,
5. exposing tolerances and iteration limits in the GUI rather than hiding them.

These design choices do not guarantee perfect identifiability, but they do strongly improve reproducibility and numerical readability.

8.2 When stricter settings are scientifically advisable

Tighter tolerances should be used in at least four situations:

- very low perfusion,
- extremely small or extremely large `P_half`,
- sharp transitions in sweep studies,
- figures intended for publication or peer review.

In such settings, small numerical differences can be physiologically amplified by the nonlinear saturation and consumption terms.

9 Diagnostic layer and inverse reconstruction

The application imports `OxygenationInput` and `alert_decision` from `oxygenation_diagnostic_mvp.py`. The diagnostic module converts blood-gas variables and a local sensor reading into a continuous risk score and five class probabilities.

The main GUI then offers a second step: reconstruction of a plausible Krogh cylinder from these measured or assumed quantities. This is scientifically useful because a low tissue sensor value can arise from multiple causes, including low inflow oxygen, reduced perfusion, altered hemoglobin unloading, or increased effective tissue demand.

9.1 From venous saturation to a fitting target

When the reconstruction workflow is started, the GUI transforms the optional venous saturation into a target venous PO_2 by inverting the Hill equation:

$$P_{v,target} = P_{50,eff} \left(\frac{S_v}{1 - S_v} \right)^{1/n}.$$

In the code, the saturation is clamped to the interval $[0.01, 0.99]$ before this computation to avoid singular values.

9.2 Joint fitting of perfusion and mitochondrial demand

The scientifically most important inverse function is `_fit_joint_krogh_parameters(...)`. It performs a genuine two-parameter search over:

- perfusion factor candidates generated by `np.geomspace(0.35, 5.0, 15)` plus the reference value 1.0,
- `P_half` candidates generated by `np.geomspace(0.05, 300.0, 22)` plus the reference value 1.0.

For each candidate pair, the code solves the full mechanistic oxygen model and evaluates

$$J = |PO_{2,sensor}^{sim} - PO_{2,sensor}^{target}| + w_v |P_v^{sim} - P_v^{target}| + 0.10 |\ln(perf)| + 0.02 |\ln(P_{half})|.$$

This objective function explains exactly how the fitting behaves:

1. sensor mismatch is the primary target,
2. venous mismatch is a secondary constraint weighted by w_v,
3. logarithmic penalties discourage implausibly extreme parameters.

9.3 Why different venous weights are used

If the venous saturation field is left blank, or remains at the default value of 75%, the code assigns `venous_weight = 0.15`. If the user provides a real venous saturation value, the weight becomes `0.55`. This distinction is important: the model assumes that a default value is only a soft hint, whereas an explicitly entered value should influence the fit more strongly.

9.4 How fit quality is judged

A fit warning is raised when the sensor error exceeds 6 mmHg, or when the venous error exceeds 4 mmHg under a strongly weighted venous constraint. Consequently, a "good fit" in this workflow means more than a low objective value. It also means that the solution remains physiologically plausible and that the reconstruction is not driven by an extreme parameter combination.

9.5 What the optimized diagnostic workflow now reports

An important recent improvement is that the diagnostic section no longer stops at a single alert color. The current verified workflow now reports:

1. the dominant feature-level drivers of the alert score,
2. a best-fit assumption summary stating which perfusion and mitoP50 adjustments were required to reproduce the measured pattern,
3. hidden hypoxic burden fractions below 1, 5, 10, 15, and 20 mmHg,
4. radius-conditioned results for 30, 50, and 100 µm tissue-cylinder assumptions,
5. a clinically oriented context note clarifying that the stronger alert may be relevant mainly if the larger or swollen-tissue geometry is clinically plausible.

This change addresses a major interpretive limitation of earlier versions. Previously, a user could see an elevated alert without immediately knowing whether it reflected a robust geometry-independent finding or whether it emerged mainly under a specific tissue-radius assumption. The present workflow makes that dependency explicit.

10 Reporting and export workflow

The documentation and export layer has also been expanded substantially. The GUI can now save:

- complete case snapshots with persistent UI state,
- structured diagnostic reports in JSON or CSV format,
- English publication reports in TXT, TEX, or PDF format.

If a Krogh reconstruction figure has been generated, the latest plot is stored and inserted automatically into the publication report. This makes the software more suitable for laboratory documentation, manuscript preparation, and retrospective review of interesting cases.

11 Libraries and their concrete roles

The numerically relevant imported libraries are the following:

- `numpy`: mesh generation, clipping, interpolation, averaging, percentiles, and array algebra,
- `scipy.integrate.solve_ivp`: adaptive integration of the capillary ODE,
- `scipy.optimize.brentq`: one-dimensional root finding for the auxiliary venous fitting routine,
- `pandas`: result export and tabular template generation,
- `matplotlib`: scientific plotting for single runs and sweeps.

The key point is that no opaque machine-learning package determines the mechanistic simulation outcome. The calculation remains almost entirely reproducible from a small set of explicit NumPy and SciPy calls.

12 Interpretation guidelines for publication use

When reporting results, one should avoid relying on a single scalar metric. A robust interpretation should usually combine:

1. venous capillary PO_2 as a measure of transport reserve,
2. the 5th tissue percentile as a robust low-oxygen descriptor,
3. hypoxic area fractions for spatial burden,
4. the fitted perfusion factor and `P_half` for mechanistic interpretation.

In particular, a very low tissue signal does not automatically imply a unique pathophysiological cause. The inverse solution is model-based and therefore should be read as the most plausible explanation within the assumptions of the current Krogh framework.

13 Limitations

The current implementation is scientifically strong, but intentionally reduced. The main limitations are:

- no explicit capillary network architecture,

- no time-dependent pulsatility,
- no formal statistical uncertainty intervals on inferred parameters; the current GUI now reports a practical near-optimal fit band for mitoP50 and perfusion, but this is not yet a validated posterior interval,
- geometric uncertainty is only partially addressed: the software now compares 30, 50, and 100 µm tissue-radius assumptions, but this remains a sensitivity analysis rather than a full morphometric model,
- no formal parameter identifiability proof,
- a transparent grid search rather than a high-dimensional Bayesian inverse solver.

These limitations do not invalidate the program. They define its scope: interpretable, mechanistic, and computationally controllable micro-oxygenation analysis.

14 Conclusion

The code in `krogh_GUI.py` implements a coherent chain from physiological transport theory to numerical simulation and inverse diagnostic interpretation. The mathematical foundations are not merely mentioned in the program; they are encoded in clearly named functions, solved with explicit numerical routines, and exposed through user-adjustable parameters. This makes the software particularly well suited for research-oriented and publication-adjacent work where the provenance of each result must remain explainable.